

ID1217 Concurrent Programming

Report - Matrix operations & Quicksort

Name: Erik Sevelin, Danni Norén

Email: esevelin@kth.se, dannin@kth.se

Compute the Sum, Min, and Max of Matrix Elements

A Task description

In part A the goal is to find the maximum and minimum value and the position in the matrix using X and Y coordinates as well as the sum which was implemented in the original code.

Method

Here we will use Pthreads to compute these while using a barrier mechanism to synchronize the threads before computing the final result to make sure that all threads are done before performing the final computation and getting correct results.

This was done using arrays holding all the minimum and maximum values found by the workers(threads) as well as their positions, for this computation, we initialized the first element as the max/min values and using a for loop all the elements were compared to the the max/min value in order to determine the thread's min/max value. These values were then stored in an global array. The Barrier function was used to ensure all the threads have reached before continuing. In order to determine the final min/max values we started from the very first thread and compared the values determined by the threads using for loops to determine the final min/max values and their position.

B Task description

In this part of the assignment we can't have partial results from the threads and the main thread would determine the final results. Moreover the barrier function can't be used in this part of the assignment.

Method

In this assignment we used mutex lock for synchronization. We also used pthread_join to ensure that the main threads waits for all the workers to reach before computing the results. The workers will compute the min/max and sum the same as they did in the first assignment and they will update the global variables. Since all threads can access and change the global variables, we use mutex lock to avoid race conditions and wrong results. We have added Pthread_join in the first assignment as well but not completely curtains if it was needed.

C Task description

In this part of the assignment we will use "bag of tasks" which means each worker "picks a task" from the "bag" after finishing the task it was assigned to.

Method

Here we didn't assign fixed rows to the workers and instead we have a row counter, each worker picks one row at a time and this is ensured using the mutex lock until all rows are processed. And the final results are determined the same as in assignment B.

Analysis

Task	Execution Time(sec)	Matrix Size	Number of Threads
A	0.001633	5x5	3
B	0.000623	5x5	3
C	0.000582	5x5	3

In part A since we use barrier synchronization and all threads need to reach a certain point to be able to proceed, the execution time is slower, after removing the barrier in assignment B and using a global variable, there's less waiting time and because of that this is faster than part A. Finally part C uses "bag of task" which has the most efficient time complexity because the threads start working on a new task as soon as they are done with the previous one.

Quicksort

Summary

In summary, multithreaded Quicksort seemed to show no speedup in average time to sort k threads depending on size k . Sorting took more time, the larger k got.

Description

The purpose of this part of the report is to test the time it takes for a multithreaded Quicksort algorithm to sort large arrays (about 100 000 elements in size.)

Method

Five tests were conducted with a randomized unsorted array with a randomized length n ranging between 10 000 to 1 000 000, consisting of randomized integers ranging between 1 and 1 000 000.

The program was written in Java. Time was measured from the start of the Quicksort algorithm to its completion and was measured using `System.currentTimeMillis()` to get a general time in milliseconds.

Results

array size (n)	317 732	913 831	304 084	257 868	997 248
time (ms)	118	367	99	88	299
average time for arbitrary array size k , (ns)	371.382171k	401.605986k	325.567935k	341.259869k	299.825119k

Figure 1: test data for time (ms) based on size of array (n), and average time to sort k elements

Analysis

The randomized array lengths varied on an interval from 257 868 to 997 248. There seemed to be no correlation between size and general average speed to sort the arrays, this could be due to the small test data or the general randomness of variables, causing widely varying results. However, the sorting was in general slower for larger arrays.